
INDUSTRY-GRADE INTERVIEW PREPARATION

SystemVerilog

Practical Interview Questions

*100+ Real Questions for
Verification & RTL Design Engineers*

Topics Covered:

SystemVerilog Basics • OOP • Constrained Randomization • SVA
Functional Coverage • UVM Concepts • Interfaces • Clocking Blocks
Processes • Mailboxes • Packages • Debugging

Author:

Kittu K Patel

This book is intended for educational and interview preparation purposes.

Contents

1	SystemVerilog Basics	1
1.1	Fundamental Concepts	1
2	Data Types and Variables	5
2.1	Built-in Data Types	5
3	Functions and Tasks	9
3.1	Functions vs Tasks	9
4	OOP Concepts in SystemVerilog	12
4.1	Classes and Objects	12
5	Constrained Randomization	17
5.1	Random Variables and Constraints	17
6	SystemVerilog Assertions (SVA)	22
6.1	Immediate and Concurrent Assertions	22
7	Functional Coverage	26
7.1	Covergroups and Coverpoints	26

8	Interfaces and Clocking Blocks	30
8.1	Interface Construct	30
9	Packages, Mailboxes, and Semaphores	33
9.1	Packages	33
9.2	Mailboxes	34
9.3	Semaphores	35
10	Process Control	37
10.1	Fork-Join Constructs	37
11	Testbench Architecture	40
11.1	UVM Testbench Structure	40
12	Common Debugging Questions	44
12.1	RTL and Testbench Debugging	44
13	Additional Interview Questions	51
13.1	Mixed Concept Questions	51
	Interview Quick Reference	60

1.1 Fundamental Concepts

Q1. What is SystemVerilog and how does it differ from Verilog?

Answer:

SystemVerilog (IEEE 1800) is a unified hardware description and verification language that extends Verilog (IEEE 1364). Key differences include:

- **Data Types:** SV adds `logic`, `bit`, `byte`, `int`, `shortint`, `longint`, `packed/unpacked arrays`
- **OOP:** Classes, inheritance, polymorphism, virtual methods
- **Concurrency:** `fork/join_any`, `fork/join_none`, `processes`
- **Assertions:** SVA for formal and simulation-based verification
- **Functional Coverage:** `covergroup`, `coverpoint`, `cross`
- **Constrained Randomization:** `rand`, `randc`, `constraint`
- **Interfaces:** Encapsulate port bundles and protocols
- **Programs:** Separate testbench from design (scheduling)

Q2. Explain the SystemVerilog scheduling semantics and the delta cycle.

Answer:

SystemVerilog uses a stratified event scheduler with distinct regions per time step:

1. **Active:** Evaluate blocking assignments, **always** blocks, module instantiation
2. **NBA (Non-Blocking Assignment):** Update LHS of non-blocking assignments
3. **Observed:** Evaluate clocking block inputs, properties
4. **Re-Active:** Program block execution (**program/endprogram**)
5. **Re-NBA:** Update NBA in program blocks
6. **Postponed:** **\$monitor**, **\$strobe**

A **delta cycle** is a zero-time iteration that resolves combinational feedback before advancing simulation time.

```

1 module delta_ex;
2   logic a, b, c;
3   assign b = a;      // evaluates in Active region
4   assign c = b;      // c updated after b -- delta cycle
5   initial begin
6     a = 0;
7     #1 a = 1;
8     // At time 1: a=1 (delta0), b=1 (delta1), c=1 (delta2)
9     $display("a=%b b=%b c=%b", a, b, c);
10  end
11 endmodule

```

Listing 1.1: Delta cycle example

Q3. What is the difference between wire and logic in SystemVerilog?

Answer:

- **wire:** Net type — driven by continuous assignments or module ports. Cannot hold state. Multiple drivers resolved by **wand/wor/tri** rules.
- **logic:** Variable type — can be driven by **always** blocks, initial blocks, or continuous assigns. Cannot have multiple drivers (unlike **wire**). It is the preferred single-driver type in SV.

Rule: Use **wire** for multi-driver nets; use **logic** everywhere else.

```

1 module wire_vs_logic (input logic clk, input logic d, output

```

```

1    logic q);
2    logic internal;           // OK: driven by always_ff below
3    wire bus_line;           // OK: driven by tri-state buffer
4
5    always_ff @(posedge clk)
6        internal <= d;
7
8    assign q = internal;
9    assign bus_line = (internal) ? d : 1'bz; // tri-state
10 endmodule

```

Listing 1.2: wire vs logic

Q4. What are the four-value logic states in SystemVerilog?**Answer:**

SystemVerilog uses the **4-state** logic system (for logic, reg, wire):

- **0** — Logic low / false
- **1** — Logic high / true
- **X** — Unknown (uninitialized flip-flop, bus contention, don't-care)
- **Z** — High impedance (tri-state, undriven net)

Two-state types (bit, byte, int, etc.) only have **0** and **1**, making them faster for simulation but unable to detect uninitialized conditions.

Q5. What is the difference between always_ff, always_comb, and always_latch?**Answer:**

These are **specialized procedural blocks** introduced in SystemVerilog:

- **always_ff**: For sequential logic. Requires explicit clock edge. Tools issue warnings if combinational logic is inferred. Sensitivity list is implicit.
- **always_comb**: For combinational logic. Automatically builds sensitivity list (no missing signal bugs). Executes once at time 0.
- **always_latch**: For level-sensitive latches. Tools warn if no latch is inferred. Sensitivity list is implicit.

```

1 // Sequential register
2 always_ff @(posedge clk or negedge rst_n) begin
3     if (!rst_n) q <= '0;

```

```
4     else          q <= d;
5 end
6
7 // Combinational logic (auto sensitivity list)
8 always_comb begin
9     y = a & b | c;    // no need to list a,b,c in sensitivity
10 end
11
12 // Latch (transparent when en=1)
13 always_latch begin
14     if (en) q = d;
15 end
```

Listing 1.3: Specialized always blocks

★ **Interview Tip:** Interviewers love asking `always_ff` vs `always_comb`. Always mention that `always_comb` automatically infers the sensitivity list, eliminating a common RTL bug in traditional Verilog.

Chapter 2

1

2.1 Built-in Data Types

Q6. Explain packed vs unpacked arrays in SystemVerilog.

Answer:

- **Packed array:** Dimensions are declared *before* the variable name. Stored as a contiguous set of bits. Can be used as a vector in expressions.
- **Unpacked array:** Dimensions are declared *after* the variable name. Each element is stored separately. Cannot be directly used as a single vector.

```
1 logic [3:0][7:0] packed_2d;    // Packed: 32-bit contiguous
2                                // packed_2d[0] is 8 bits
3 logic [7:0] unpacked_2d [4];  // Unpacked: 4 x 8-bit
4                                elements
5
6 // Packed can be used in expressions
7 logic [31:0] flat = packed_2d; // Legal!
8
9 // Dynamic array (unpacked)
10 int dyn_arr[];
11 dyn_arr = new[8];              // Allocate 8 elements
12 dyn_arr.delete();              // Free memory
13
14 // Associative array
15 int assoc[string];
16 assoc["key1"] = 10;
```

```

17 // Queue
18 int q[$];
19 q.push_back(5);
20 q.push_front(1);
21 int val = q.pop_front();

```

Listing 2.1: Packed vs Unpacked arrays

Q7. What is the difference between int and integer in SystemVerilog?

Answer:

- **integer**: Verilog legacy type — **4-state**, 32-bit, signed. Can hold X/Z values.
- **int**: SystemVerilog type — **2-state**, 32-bit, signed. No X/Z, faster simulation.
- **bit**: 2-state, 1-bit, unsigned.
- **logic**: 4-state, 1-bit, can be any width.

Use **int** for loop counters and arithmetic. Use **logic** for RTL signals that may be X/Z.

```

1 integer i4 = 'x;      // 4-state: holds X
2 int     i2 = 'x;      // 2-state: X becomes 0!
3 bit     b  = 1'b1;     // 2-state: x becomes 0
4
5 // Size reference
6 byte     b8;    // 8-bit, 2-state, signed
7 shortint s16;   // 16-bit, 2-state, signed
8 int      i32;   // 32-bit, 2-state, signed
9 longint  l64;   // 64-bit, 2-state, signed

```

Listing 2.2: Data type comparison

Q8. Explain typedef, struct, and enum in SystemVerilog.

Answer:

- **typedef**: Creates a user-defined type alias
- **struct**: Groups related signals; can be **packed** (contiguous bits) or **unpacked**
- **enum**: Named set of integer constants; improves readability and catches invalid

assignments

```
1 // Enum for FSM states
2 typedef enum logic [1:0] {
3     IDLE    = 2'b00,
4     READ    = 2'b01,
5     WRITE   = 2'b10,
6     DONE    = 2'b11
7 } state_t;
8
9 state_t current_state, next_state;
10
11 // Packed struct for AXI-like transaction
12 typedef struct packed {
13     logic [31:0] addr;
14     logic [31:0] data;
15     logic [3:0]  strobe;
16     logic        valid;
17     logic        ready;
18 } axi_beat_t;
19
20 axi_beat_t beat;
21 beat.addr  = 32'hDEAD_BEEF;
22 beat.valid = 1'b1;
23
24 // Using typedef
25 typedef logic [7:0] byte_t;
26 byte_t my_byte = 8'hFF;
```

Listing 2.3: typedef, struct, enum usage

Q9. What is a string type in SystemVerilog? How is it different from a string literal?

Answer:

The **string** type is a **dynamic, variable-length** built-in type introduced in SystemVerilog. It is not synthesizable but widely used in testbenches. Key string methods: `len()`, `substr()`, `itoa()`, `atoi()`, `toupper()`, `tolower()`, `compare()`.

```
1 string s1 = "Hello";
2 string s2 = "World";
3 string s3;
4
5 s3 = {s1, " ", s2}; // Concatenation
6 $display("Length: %0d", s3.len()); // 11
7 $display("Upper: %s", s3.toupper());
```

```
8
9 // String formatting
10 int val = 42;
11 string fmt;
12 fmt = $sformatf("Value = %0d", val);
13 $display(fmt);
14
15 // Comparison
16 if (s1.compare(s2) == 0)
17     $display("Strings are equal");
```

Listing 2.4: String operations

Q10. What are the differences between = (blocking) and <= (non-blocking) assignments?

Answer:

- **Blocking (=):** Executes immediately, updates the variable before the next statement. Used in combinational logic (`always_comb`).
- **Non-Blocking (<=):** Schedules the update to occur at end of the time step (NBA region). The RHS is evaluated immediately but the LHS is updated later. Used in sequential logic (`always_ff`).

Golden Rule: Never mix blocking and non-blocking in the same `always` block for RTL.

```
1 // WRONG: Race condition with blocking in sequential
2 always @(posedge clk) begin
3     a = b;    // Blocking: immediate update
4     b = a;    // BUG: b gets a's NEW value, not original
5 end
6
7 // CORRECT: Non-blocking for sequential (shift register)
8 always_ff @(posedge clk) begin
9     a <= b;   // Both RHS evaluated at same time
10    b <= a;   // a and b swap correctly
11 end
12
13 // Correct: Blocking in combinational
14 always_comb begin
15     temp = a & b;    // Blocking OK here
16     y     = temp | c;
17 end
```

Listing 2.5: Blocking vs Non-Blocking

3.1 Functions vs Tasks

Q11. What is the difference between a function and a task in SystemVerilog?

Answer:

Feature	Function	Task
Time consumption	Zero time	Can consume time
Timing controls	Not allowed	Allowed (<code>#</code> , <code>@</code> , <code>wait</code>)
Return value	Single value	Via output/inout args
Can call tasks	No	Yes
Can call functions	Yes	Yes
Synthesizable	Yes (with restrictions)	Typically No

```

1 // Function: combinational, no time, returns value
2 function automatic logic [7:0] parity_calc(
3     input logic [7:0] data);
4     return ^data; // XOR reduction
5 endfunction
6
7 // Task: can have delays, multiple outputs
8 task automatic read_mem(
9     input logic [31:0] addr,
10    output logic [31:0] data,
11    output logic      error);
12
13    @(posedge clk); // Wait for clock
14    #1;             // Propagation delay

```

```

15     data    = mem[addr];
16     error   = (addr > MAX_ADDR);
17 endtask
18
19 // Usage
20 logic [7:0] p;
21 logic [31:0] rdata;
22 logic        err;
23
24 p = parity_calc(8'hA5);
25 read_mem(32'h100, rdata, err);

```

Listing 3.1: Function and Task examples

Q12. What does the automatic keyword do in a function or task?**Answer:**

By default, functions and tasks have **static** storage — variables retain their values between calls and are shared across all concurrent invocations. The **automatic** keyword gives the function/task **dynamic (stack-based) storage**:

- Each call gets its own private copy of local variables
- Required for recursive functions
- Required for reentrant tasks used in fork/join
- Classes are always automatic

```

1 // Static: shared variable (dangerous in fork)
2 task static_task(input int id);
3     static int call_count = 0; // Shared!
4     call_count++;
5     $display("Call %0d from task %0d", call_count, id);
6 endtask
7
8 // Automatic: recursive-safe
9 function automatic int factorial(input int n);
10     if (n <= 1) return 1;
11     return n * factorial(n - 1); // Recursive: needs
    automatic
12 endfunction
13
14 // Automatic task safe in fork
15 task automatic drive_bus(input int thread_id);
16     int local_var = thread_id; // Private per invocation
17     repeat(10) @(posedge clk);
18     $display("Thread %0d done", local_var);
19 endtask

```

```
20
21 initial begin
22     fork
23         drive_bus(1); // Each gets own local_var
24         drive_bus(2);
25         drive_bus(3);
26     join
27 end
```

Listing 3.2: Static vs Automatic storage

Q13. What is a void function in SystemVerilog?**Answer:**

A void function is a function that does not return a value. It can be called as a statement (like a task) but cannot consume simulation time. Commonly used for display utilities, callbacks, and covergroup sampling.

```
1 function automatic void print_transaction(
2     input logic [31:0] addr,
3     input logic [31:0] data,
4     input string      op);
5
6     $display("[%0t] %s: addr=0x%08h data=0x%08h",
7             $time, op, addr, data);
8 endfunction
9
10 // Called as a statement (no return value needed)
11 print_transaction(32'hABCD, 32'h1234, "WRITE");
```

Listing 3.3: void function example

4.1 Classes and Objects

Q14. What are the four pillars of OOP in SystemVerilog?

Answer:

1. **Encapsulation:** Bundle data and methods in a class; use `local/protected` to restrict access
2. **Inheritance:** `extends` keyword; child class inherits parent properties/methods
3. **Polymorphism:** virtual methods allow derived class overrides called through base class handle
4. **Abstraction:** pure virtual methods define an interface without implementation

```
1 // Base class (abstraction)
2 virtual class base_transaction;
3     rand logic [31:0] addr;
4     rand logic [31:0] data;
5
6     pure virtual function string get_name();
7
8     function void print();
9         $display("%s: addr=%h data=%h", get_name(), addr, data);
10    endfunction
11 endclass
```



```

12
13 // Derived class (inheritance + polymorphism)
14 class write_transaction extends base_transaction;
15     rand logic [3:0] byte_enable;
16
17     // Override pure virtual
18     function string get_name();
19         return "WRITE";
20     endfunction
21
22     constraint valid_be {
23         byte_enable inside {4'b0001, 4'b0011, 4'b1111};
24     }
25 endclass
26
27 // Usage
28 base_transaction btx;
29 write_transaction wtx = new();
30 btx = wtx;                // Polymorphic assignment
31 btx.print();              // Calls write_transaction::get_name()

```

Listing 4.1: OOP fundamentals in SV

Q15. Explain virtual vs pure virtual methods.**Answer:**

- **virtual method:** Has a default implementation in base class. Derived class *may* override it. Base class may or may not be abstract.
- **pure virtual method:** No implementation in base class. Derived class *must* override it. The containing class is implicitly **virtual** (abstract) and *cannot* be instantiated.

```

1 virtual class driver_base;
2     // Pure virtual: must be overridden
3     pure virtual task drive(input logic [31:0] data);
4
5     // Virtual: has default, may be overridden
6     virtual function void report();
7         $display("Generic driver report");
8     endfunction
9 endclass
10
11 class apb_driver extends driver_base;
12     // Must implement pure virtual
13     task drive(input logic [31:0] data);

```

```

14   @(posedge clk);
15   psel  = 1;
16   penable = 0;
17   paddr = current_addr;
18   pwidth = data;
19   @(posedge clk);
20   penable = 1;
21   @(posedge clk);
22   psel = 0;
23   endtask
24
25   // Optionally override virtual
26   function void report();
27       $display("APB Driver: %0d transactions", tx_count);
28   endfunction
29   endclass

```

Listing 4.2: virtual vs pure virtual

Q16. What is the difference between `new()` deep copy and shallow copy?

Answer:

- **Shallow copy (=):** Copies only the handle (pointer). Both variables point to the *same* object in memory. Modifying one affects the other.
- **Deep copy (`new obj2 = new(obj1)`):** Creates a completely new object with copied values. Modifying one does *not* affect the other. For nested objects, you must recursively deep copy.

```

1  class packet;
2      int data;
3      int id;
4      function new(int d=0, int i=0);
5          data = d; id = i;
6      endfunction
7      // Deep copy constructor
8      function packet copy();
9          copy = new();
10         copy.data = this.data;
11         copy.id   = this.id;
12     endfunction
13 endclass
14
15 packet p1 = new(10, 1);
16 packet p2, p3;

```

```

17
18 // Shallow copy: p2 and p1 point to same object
19 p2 = p1;
20 p2.data = 99;
21 $display(p1.data); // Prints 99! Same object.
22
23 // Deep copy: independent objects
24 p3 = p1.copy();
25 p3.data = 77;
26 $display(p1.data); // Prints 10. Different object.

```

Listing 4.3: Shallow vs Deep copy

Q17. What is \$cast and when do you use it in SystemVerilog?**Answer:**

\$cast performs a **dynamic type cast** from a base class handle to a derived class handle (downcasting). It returns 1 on success and 0 on failure (unlike static cast which would generate an error). Used when you have a base class handle that may contain a derived class object and you need to access derived-class-specific members.

```

1 class animal;
2     virtual function void speak(); endfunction
3 endclass
4
5 class dog extends animal;
6     int tricks = 5;
7     function void speak(); $display("Woof!"); endfunction
8     function void fetch(); $display("Fetching!"); endfunction
9 endclass
10
11 class cat extends animal;
12     function void speak(); $display("Meow!"); endfunction
13 endclass
14
15 animal a;
16 dog    d = new();
17 cat    c_obj;
18
19 a = d; // Upcast: always safe
20
21 // Downcast using $cast
22 if (!$cast(c_obj, a)) begin
23     $display("Cast failed: 'a' is not a cat!");
24 end
25
26 dog d2;

```

```
27 if ($cast(d2, a)) begin
28     d2.fetch();    // Success: a was holding a dog
29     $display("Tricks: %0d", d2.tricks);
30 end
```

Listing 4.4: \$cast for downcasting

Q18. Explain handles and the null keyword in SystemVerilog classes.

Answer:

A **handle** is a reference (pointer-like) to a class object. Before calling `new()`, a handle is **null**. Accessing members of a null handle causes a runtime fatal error. Always check handles before use in verification code.

```
1 class my_packet;
2     int data;
3 endclass
4
5 my_packet pkt;    // Handle declared; currently null
6
7 // Runtime fatal: pkt is null!
8 // pkt.data = 5;    // DO NOT do this
9
10 // Proper usage
11 pkt = new();    // Allocate object
12 pkt.data = 42;
13
14 // Null check
15 if (pkt == null)
16     $fatal(1, "Packet handle is null!");
17 else
18     $display("Data = %0d", pkt.data);
19
20 // Free (let GC collect)
21 pkt = null;    // No longer referenced; GC will reclaim
```

Listing 4.5: Handle and null checks

5.1 Random Variables and Constraints

Q19. What is the difference between `rand` and `randc`?

Answer:

- **rand:** Uniformly random. A variable can take the same value in consecutive calls to `randomize()`.
- **randc:** Random *cyclic*. The variable cycles through all possible values (like shuffling a deck) before repeating any value. Ensures complete coverage without repetition within a cycle.

```
1 class rand_demo;
2   rand logic [2:0] r;    // Random: may repeat values
3   randc logic [2:0] rc;  // Cyclic: visits 0-7 once per
   cycle
4
5   function void show(int n);
6     repeat(n) begin
7       void'(this.randomize());
8       $display("rand=%0d  randc=%0d", r, rc);
9     end
10  endfunction
11 endclass
12
13 // Typical output for 8 iterations:
14 // rand:  3 1 3 7 2 1 5 6  (may repeat 1,3)
15 // randc: 4 7 0 3 6 1 5 2  (all unique in first 8)
```

Listing 5.1: rand vs randc behavior

Q20. How do you enable/disable constraints in SystemVerilog?**Answer:**

Use `constraint_name.constraint_mode(0/1)` to disable/enable a named constraint block. Alternatively, use `object.randomize()` with `{...}` for inline/ad-hoc constraints. Use `rand_mode(0)` on a variable to turn off randomization entirely.

```

1 class transaction;
2     rand logic [7:0] data;
3     rand logic [7:0] addr;
4
5     constraint data_range { data inside {[0:127]}; }
6     constraint addr_align { addr[1:0] == 2'b00; }
7 endclass
8
9 transaction t = new();
10
11 // Normal randomize
12 t.randomize();
13
14 // Disable a constraint
15 t.data_range.constraint_mode(0);
16 t.randomize(); // data can now be 0-255
17
18 // Re-enable
19 t.data_range.constraint_mode(1);
20
21 // Inline constraint override
22 t.randomize() with {
23     data == 8'hFF; // Force specific value
24     addr inside {[8'h00 : 8'h3F]};
25 };
26
27 // Turn off rand for a variable
28 t.data.rand_mode(0);
29 t.randomize(); // data keeps its current value

```

Listing 5.2: Constraint control

Q21. What is the solve...before constraint? When is it used?

Answer:

solve A before B biases the constraint solver to **first randomize A**, then solve B based on A's value. It does NOT change the mathematical distribution but affects the **probability distribution** when there is a conditional constraint linking A and B. Without it, the solver may produce a uniform distribution over the joint space, which may under-represent certain valid scenarios.

```

1 class packet;
2     rand bit          is_error;
3     rand logic [7:0]  error_code;
4
5     // Without solve..before: is_error=1 probability may be
6     // because only some error_code values correspond to
7     // is_error=1
8     constraint err_link {
9         solve is_error before error_code;
10        if (is_error)
11            error_code inside {[8'h80 : 8'hFF]};
12        else
13            error_code inside {[8'h00 : 8'h7F]};
14    }
15 endclass
16 // Now is_error=0 and is_error=1 each occur ~50% of the time

```

Listing 5.3: solve...before example

Q22. Explain the inside operator in constraints.**Answer:**

The **inside** operator checks if a value is within a set or range. It can be used in both constraints and expressions. Supports ranges with [lo:hi], individual values, and mixed lists.

```

1 class burst_config;
2     rand logic [3:0] length;
3     rand logic [1:0] size;
4     rand logic [7:0] addr;
5
6     // Inside with ranges and values
7     constraint valid_length {
8         length inside {1, 2, 4, 8, [12:15]};
9     }
10
11    // Inside with size values
12    constraint valid_size {

```

```

13     size inside {2'b00, 2'b01, 2'b10};
14     // Same as: size != 2'b11
15 }
16
17 // Address alignment based on size
18 constraint addr_align {
19     if (size == 2'b01)      addr[0] == 0; // 2-byte
        aligned
20     else if (size == 2'b10) addr[1:0] == 0; // 4-byte
        aligned
21 }
22 endclass
23
24 // inside in expressions (non-constraint context)
25 logic [3:0] data;
26 if (data inside {4'h0, 4'hF, [4'hA:4'hC]})
27     $display("Data is in set");

```

Listing 5.4: inside operator usage

Q23. What causes a constraint solver failure and how do you debug it?**Answer:**

A solver failure (`randomize()` returns 0) occurs when constraints are **unsatisfiable** — i.e., there is no valid value that satisfies all constraints simultaneously. Common causes:

- Contradictory constraints (e.g., $a > 10$ and $a < 5$)
- Over-constrained system (too many tight constraints)
- Incorrect use of inheritance with conflicting parent constraints

Debug by: disabling constraints one by one, using `$display` to trace, simplifying the constraint block, or using solver analysis tools.

```

1 class my_obj;
2     rand int a;
3     constraint c1 { a > 10; }
4     constraint c2 { a < 5; } // Contradicts c1!
5 endclass
6
7 my_obj obj = new();
8
9 // ALWAYS check return value of randomize()
10 if (!obj.randomize()) begin
11     $fatal(1, "[%0t] randomize() FAILED for my_obj!", $time);
12 end
13

```



```
14 // Better: use assert
15 assert(obj.randomize()) else
16     $error("Randomization failed      check constraints");
17
18 // Debug: isolate by disabling constraints
19 obj.c2.constraint_mode(0);
20 if (obj.randomize())
21     $display("c2 was the problem constraint");
```

Listing 5.5: Detecting and handling randomize failure

6.1 Immediate and Concurrent Assertions

Q24. What is the difference between immediate and concurrent assertions?

Answer:

- **Immediate assertions:** Evaluated at the point of execution in a procedural block, like an if statement. No clock required. Evaluated in zero time.
- **Concurrent assertions:** Evaluate **over time** based on a clock. Sampled at clock edges. Described using sequences and properties. Can span multiple cycles. Used in formal verification and simulation.

```
1 // Immediate assertion (inside procedural block)
2 always_comb begin
3     assert (data_width inside {8, 16, 32, 64})
4         else $error("Invalid data width: %0d", data_width);
5 end
6
7 // Concurrent assertion (clock-based, outside procedural)
8 // If req goes high, ack must follow within 4 cycles
9 property req_ack_check;
10    @(posedge clk) disable iff (!rst_n)
11        req |-> ##[1:4] ack;
12 endproperty
13
14 assert property (req_ack_check)
15     else $error("ACK not received within 4 cycles after REQ");
```

```

16
17 cover property (req_ack_check)
18     $display("req->ack scenario covered");

```

Listing 6.1: Immediate vs Concurrent assertions

Q25. Explain the implication operators `|->` and `|=>` in SVA.

Answer:

- **Overlapping implication** `|->`: If the antecedent matches, the consequent starts *at the same clock cycle*. “If A then *at same time* B.”
- **Non-overlapping implication** `|=>`: If the antecedent matches, the consequent starts *at the next clock cycle*. Equivalent to `|-> ##1`.

```

1  // Overlapping: grant must be high SAME cycle as req
2  property overlap_check;
3      @(posedge clk) req |-> grant;
4  endproperty
5  // req=1 at T: grant must be 1 at T
6
7  // Non-overlapping: valid must be high NEXT cycle after
   start
8  property non_overlap_check;
9      @(posedge clk) start |=> valid;
10 endproperty
11 // start=1 at T: valid must be 1 at T+1
12
13 // Multi-cycle: data valid 2 cycles after command
14 property cmd_data_check;
15     @(posedge clk) cmd_valid |-> ##2 data_valid;
16 endproperty
17
18 // Range: ack within 1 to 5 cycles
19 property ranged_ack;
20     @(posedge clk) req |-> ##[1:5] ack;
21 endproperty
22
23 assert property (overlap_check);
24 assert property (non_overlap_check);
25 assert property (cmd_data_check);
26 assert property (ranged_ack);

```

Listing 6.2: Implication operators in SVA

Q26. What are the \$rose, \$fell, \$stable, and \$past system functions in SVA?

Answer:

These are **sampled value functions** used in concurrent assertions:

- \$rose(sig): Returns 1 if LSB of sig transitioned from 0 to 1 (rising edge)
- \$fell(sig): Returns 1 if LSB of sig transitioned from 1 to 0 (falling edge)
- \$stable(sig): Returns 1 if sig did not change in the last clock cycle
- \$past(sig, n): Returns the value of sig n cycles ago (default n=1)

```

1 // Check that data is stable when valid is low
2 property data_stable_when_idle;
3   @(posedge clk) !valid |-> $stable(data);
4 endproperty
5
6 // Check rising edge of req followed by ack
7 property req_handshake;
8   @(posedge clk) $rose(req) |-> ##[1:4] $rose(ack);
9 endproperty
10
11 // Check data matches what was registered 2 cycles ago
12 property pipeline_check;
13   logic [7:0] captured;
14   @(posedge clk) (1, captured = data_in) |->
15     ##2 (data_out == $past(data_in, 2));
16 endproperty
17
18 // Check valid deasserted after one cycle
19 property valid_pulse;
20   @(posedge clk) $rose(valid) |> $fell(valid);
21 endproperty
22
23 assert property (data_stable_when_idle);
24 assert property (req_handshake);
25 assert property (valid_pulse);

```

Listing 6.3: Sampled value functions in SVA

Q27. What is the disable iff clause in SVA?

Answer:

disable iff (condition) acts as an **asynchronous reset** for a concurrent

assertion. When the condition is true, the property evaluation is vacuously true — the assertion is not checked. Typically used to suppress assertions during reset.

```
1 // Suppress assertion during active-low reset
2 property fifo_not_overflow;
3   @(posedge clk) disable iff (!rst_n || !enable)
4     !full |-> ##1 !overflow;
5 endproperty
6
7 // Without disable iff, X-propagation during reset
8 // would cause false assertion failures
9 assert property (fifo_not_overflow)
10    else $error("FIFO overflow detected!");
11
12 // Multiple disable conditions
13 property bus_protocol;
14   @(posedge clk) disable iff (reset || test_mode || flush)
15     req |-> ##[1:8] ack;
16 endproperty
```

Listing 6.4: disable iff usage

7.1 Covergroups and Coverpoints

Q28. What is functional coverage and how does it differ from code coverage?

Answer:

- **Code Coverage:** Measures which lines/branches/expressions of the *RTL code* were exercised. Structural metric. Automatically extracted. Does NOT measure whether the design was tested correctly.
- **Functional Coverage:** Measures whether specific *design functionality scenarios* were tested, as defined by the verification plan. Intent-driven metric. Must be manually written. Answers: “Did we test all corner cases?”

Both are needed: code coverage ensures no dead code; functional coverage ensures all features are verified.

Q29. Explain the structure of a covergroup, coverpoint, and cross.

Answer:

- **Covergroup:** Container that groups related coverage points. Sampled at a defined event (clock edge, manual `.sample()`).

- **Coverpoint:** Monitors a specific variable or expression and tracks which values (bins) have been hit.
- **Cross:** Measures all combinations of two or more coverpoints.

```

1  covergroup transaction_cg @(posedge clk);
2  // Coverpoint: track transaction type
3  cp_type: coverpoint trans_type {
4      bins read  = {READ};
5      bins write = {WRITE};
6      bins idle  = {IDLE};
7  }
8
9  // Coverpoint with ranges
10 cp_length: coverpoint burst_length {
11     bins short  = {[1:4]};
12     bins medium = {[5:16]};
13     bins long   = {[17:256]};
14 }
15
16 // Coverpoint with automatic bins
17 cp_addr: coverpoint addr[3:0]; // 16 auto bins (0-15)
18
19 // Cross coverage: all type x length combinations
20 cx_type_len: cross cp_type, cp_length {
21     // Exclude idle with non-zero length
22     ignore_bins idle_long = binsof(cp_type.idle) &&
23                             binsof(cp_length.long);
24 }
25
26 // Option
27 option.per_instance = 1;
28 option.goal = 100;
29 endgroup
30
31 transaction_cg cg_inst = new();
32 // Manual sampling
33 cg_inst.sample();

```

Listing 7.1: Covergroup structure

Q30. What are illegal_bins and ignore_bins?

Answer:

- **ignore_bins:** Excludes certain values from coverage calculation. These values will not contribute to hit counts but will not cause an error if seen.

- **illegal_bins:** Declares certain values as illegal. If an illegal value is sampled, a runtime error/warning is generated. Used for protocol violations.

```

1  covergroup opcode_cg;
2      cp_opcode: coverpoint opcode {
3          bins valid_ops[] = {ADD, SUB, MUL, AND, OR};
4
5          // These are legal but uninteresting
6          ignore_bins nop_ops = {NOP, HALT};
7
8          // These should NEVER occur in simulation
9          illegal_bins reserved = {[4'hA : 4'hF]};
10     }
11 endgroup

```

Listing 7.2: ignore_bins and illegal_bins

Q31. How do you query coverage results programmatically in SystemVerilog?

Answer:

Use the `$get_coverage()` system function or the coverage methods on covergroup instances: `get_coverage()`, `get_inst_coverage()`. These return real values 0.0–100.0.

```

1  covergroup my_cg;
2      cp_data: coverpoint data { bins lo = {[0:127]}; bins hi =
3          {[128:255]}; }
4  endgroup
5
6  my_cg cg = new();
7
8  // After simulation
9  real cov;
10 cov = cg.get_coverage(); // Overall coverage %
11 cov = cg.cp_data.get_coverage(); // Per-coverpoint
12
13 $display("Coverage = %0.2f%%", cov);
14
15 // Global coverage
16 $display("Global coverage = %0.2f%%", $get_coverage());
17
18 // Stop simulation when coverage goal met
19 initial begin
20     wait(cg.get_coverage() >= 99.9);
21     $display("Coverage goal achieved!");
22     $finish;

```



```
22 | end
```

Listing 7.3: Querying coverage

8.1 Interface Construct

Q32. What is a SystemVerilog interface and why is it used?

Answer:

An **interface** encapsulates a group of related signals (a communication channel) into a single entity. Benefits:

- **Reduces port count:** Pass one interface instead of dozens of signals
- **Modports:** Define directional views (driver vs receiver) within the interface
- **Protocol encapsulation:** Embed tasks, functions, assertions, and clocking blocks
- **Easier connection:** Reduces wiring errors in testbenches
- **Reusability:** Reuse the same interface across multiple DUTs

```
1 // Interface definition
2 interface apb_if (input logic clk, input logic rst_n);
3     logic        psel;
4     logic        penable;
5     logic [31:0] paddr;
6     logic [31:0] pwdata;
7     logic [31:0] prdata;
8     logic        pready;
9     logic        pslverr;
10    logic        pwrite;
11
12    // Modport for master (driver)
```

```

13  modport master (
14      input  clk, rst_n, prdata, pready, pslverr,
15      output psel, penable, paddr, pwidth, pwrite
16  );
17
18  // Modport for slave (DUT)
19  modport slave (
20      input  clk, rst_n, psel, penable, paddr, pwidth, pwrite,
21      output prdata, pready, pslverr
22  );
23
24  // Assertion inside interface
25  property no_error_when_sel_low;
26      @(posedge clk) !psel |-> !pslverr;
27  endproperty
28  assert property (no_error_when_sel_low);
29 endinterface
30
31 // DUT using interface
32 module apb_slave (apb_if.slave bus);
33     always_ff @(posedge bus.clk) begin
34         if (bus.psel && bus.penable && !bus.pwrite)
35             bus.prdata <= mem[bus.paddr];
36     end
37 endmodule
38
39 // Testbench
40 module tb;
41     logic clk = 0, rst_n = 1;
42     always #5 clk = ~clk;
43
44     apb_if apb_bus (.clk(clk), .rst_n(rst_n));
45     apb_slave dut (.bus(apb_bus.slave));
46 endmodule

```

Listing 8.1: Interface definition and usage

Q33. What is a clocking block and why is it important in testbenches?

Answer:

A **clocking block** defines the **synchronization** between testbench and DUT. It specifies:

- The sampling edge for inputs (e.g., 1step before posedge = end of time step)
- The drive skew for outputs (e.g., #2ns after posedge)

This prevents **race conditions** between the testbench driving signals and the DUT sampling them. Without a clocking block, a testbench driving at the same

posedge as the DUT can cause non-deterministic behavior.

```
1 interface axi_if (input logic clk);
2     logic        valid;
3     logic        ready;
4     logic [31:0] data;
5
6     clocking driver_cb @(posedge clk);
7         default input  #1step;    // Sample just before clock
            edge
8         default output #2ns;      // Drive 2ns after clock edge
9         output valid, data;
10        input  ready;
11    endclocking
12
13    clocking monitor_cb @(posedge clk);
14        default input #1step;
15        input valid, ready, data;
16    endclocking
17
18    modport driver  (clocking driver_cb,  input clk);
19    modport monitor (clocking monitor_cb, input clk);
20 endinterface
21
22 // Driver using clocking block
23 task drive_transfer(axi_if.driver bus, logic [31:0] d);
24     bus.driver_cb.valid <= 1;
25     bus.driver_cb.data  <= d;
26     @(bus.driver_cb);    // Wait for clock
27     while (!bus.driver_cb.ready) @(bus.driver_cb);
28     bus.driver_cb.valid <= 0;
29 endtask
```

Listing 8.2: Clocking block in interface

Chapter 9

1

9.1 Packages

Q34. What is a package in SystemVerilog and what can it contain?

Answer:

A **package** is a namespace that groups reusable definitions. Unlike modules, it does not represent hardware. A package can contain:

- typedef, enum, struct
- Parameters and local parameters
- Functions and tasks
- Class definitions
- Import statements from other packages

Packages prevent name collisions and improve code reuse across projects.

```
1 package axi_pkg;  
2   // Parameters  
3   parameter AXI_ADDR_WIDTH = 32;  
4   parameter AXI_DATA_WIDTH = 64;  
5  
6   // Types  
7   typedef logic [AXI_ADDR_WIDTH-1:0] axi_addr_t;  
8   typedef logic [AXI_DATA_WIDTH-1:0] axi_data_t;  
9  
10  // Enum  
11  typedef enum logic [1:0] {
```

```
12     FIXED = 2'b00, INCR = 2'b01, WRAP = 2'b10
13 } axi_burst_t;
14
15 // Function
16 function automatic axi_addr_t align_addr(
17     input axi_addr_t addr, input int size);
18     return addr & ~((1 << size) - 1);
19 endfunction
20 endpackage
21
22 // Import in module
23 module axi_master
24     import axi_pkg::*; // Wildcard import
25     (* ports *);
26     axi_addr_t base_addr;
27     axi_burst_t burst_type = axi_pkg::INCR; // Explicit
28     import
29
29     always_comb
30         base_addr = align_addr(raw_addr, 3); // 8-byte align
31 endmodule
```

Listing 9.1: Package definition and import

9.2 Mailboxes

Q35. What is a mailbox in SystemVerilog and how is it used?

Answer:

A **mailbox** is a built-in inter-process communication (IPC) mechanism — a FIFO queue for passing data between threads. Key methods:

- `put(item)`: Blocking write (waits if mailbox is full — bounded only)
- `get(item)`: Blocking read (waits until data available)
- `try_put()`: Non-blocking write; returns 0 if full
- `try_get()`: Non-blocking read; returns 0 if empty
- `peek(item)`: Read without removing
- `num()`: Returns number of items currently in mailbox

```
1 class transaction;
2     rand logic [31:0] data;
3     rand logic [31:0] addr;
```

```

4  endclass
5
6  // Shared mailbox (typically in environment)
7  mailbox #(transaction) gen2drv = new(10); // Bounded: max 10
8
9  // Generator thread
10 task generator();
11     transaction t;
12     repeat(100) begin
13         t = new();
14         assert(t.randomize());
15         gen2drv.put(t); // Blocking: waits if full
16         $display("Generated addr=%h", t.addr);
17     end
18 endtask
19
20 // Driver thread
21 task driver();
22     transaction t;
23     forever begin
24         gen2drv.get(t); // Blocking: waits for data
25         // Drive to DUT
26         @(posedge clk);
27         dut_addr = t.addr;
28         dut_data = t.data;
29         dut_valid = 1;
30         @(posedge clk);
31         dut_valid = 0;
32     end
33 endtask
34
35 // Launch both
36 initial fork
37     generator();
38     driver();
39 join

```

Listing 9.2: Mailbox usage between generator and driver

9.3 Semaphores

Q36. What is a semaphore in SystemVerilog? Give a practical example.

Answer:

A **semaphore** is a synchronization primitive with a bucket of tokens. Used for **mutual exclusion** (mutex) and resource arbitration. Key methods:

- **new(N)**: Creates semaphore with N initial tokens
- **get(N)**: Acquire N tokens (blocks if fewer than N available)
- **put(N)**: Release N tokens
- **try_get(N)**: Non-blocking acquire; returns 0 if unavailable

```

1  // Shared bus protected by semaphore
2  semaphore bus_lock = new(1);  // 1 token = mutex
3
4  // Thread 1: CPU access
5  task cpu_access(logic [31:0] addr, data);
6      bus_lock.get(1);          // Acquire lock
7      $display("[%0t] CPU acquired bus", $time);
8
9      @(posedge clk); bus_addr = addr;
10     @(posedge clk); bus_data = data;
11     @(posedge clk);          // Complete transaction
12
13     bus_lock.put(1);          // Release lock
14     $display("[%0t] CPU released bus", $time);
15 endtask
16
17 // Thread 2: DMA access
18 task dma_access(logic [31:0] addr);
19     bus_lock.get(1);          // Waits if CPU holds bus
20     $display("[%0t] DMA acquired bus", $time);
21     // ... DMA burst ...
22     bus_lock.put(1);
23 endtask
24
25 // Launch concurrent threads
26 initial fork
27     cpu_access(32'h100, 32'hDEAD);
28     dma_access(32'h200);
29 join

```

Listing 9.3: Semaphore for bus arbitration

10.1 Fork-Join Constructs

Q37. Explain fork/join, fork/join_any, and fork/join_none.

Answer:

- **fork/join:** Wait for *ALL* spawned threads to complete before proceeding.
- **fork/join_any:** Wait for *ANY ONE* spawned thread to complete, then continue. Remaining threads keep running.
- **fork/join_none:** Start all threads and continue *immediately* without waiting. All threads run in background.

```

1 task t1(); #10; $display("[%0t] T1 done", $time); endtask
2 task t2(); #5;  $display("[%0t] T2 done", $time); endtask
3 task t3(); #20; $display("[%0t] T3 done", $time); endtask
4
5 // fork/join: wait for all (completes at T=20)
6 initial begin
7     fork t1(); t2(); t3(); join
8     $display("[%0t] All done", $time);    // At T=20
9 end
10
11 // fork/join_any: wait for first (continues at T=5)
12 initial begin
13     fork t1(); t2(); t3(); join_any
14     $display("[%0t] One done", $time);    // At T=5 (t2
                                         finished)

```

```

15 end
16
17 // fork/join_none: no wait (continues at T=0)
18 initial begin
19     fork t1(); t2(); t3(); join_none
20     $display("[%0t] Launched all", $time); // At T=0
21     // T1,T2,T3 still running in background
22     wait fork; // Optional: wait for all to finish later
23 end

```

Listing 10.1: fork/join variants

Q38. How do you kill or suspend a process in SystemVerilog?**Answer:**

Use the `process` class to manage running threads. The `process` class provides: `self()`, `status()`, `kill()`, `suspend()`, `resume()`, `await()`.

```

1 // Process control with process class
2 process p1_handle, p2_handle;
3
4 initial begin
5     fork
6         begin : thread1
7             p1_handle = process::self();
8             repeat(100) @(posedge clk);
9             $display("Thread1 completed");
10        end
11
12        begin : thread2
13            p2_handle = process::self();
14            @(posedge clk);
15            p1_handle.suspend(); // Pause thread1
16            #50ns;
17            p1_handle.resume(); // Resume thread1
18        end
19
20        begin : watchdog
21            #10us; // Global timeout
22            if (p1_handle.status() != process::FINISHED) begin
23                $error("Watchdog: killing thread1!");
24                p1_handle.kill();
25            end
26        end
27    join_any
28
29    disable fork; // Kill all remaining threads
30 end

```

Listing 10.2: Process class for thread control

Q39. What is the wait fork statement?**Answer:**

`wait fork` causes the calling thread to block until **all child threads** (spawned by `fork/join_none` or `fork/join_any`) from the *current scope* have completed. It is commonly used to synchronize after launching background threads.

```
1 initial begin
2     // Launch many driver threads
3     for (int i = 0; i < NUM_AGENTS; i++) begin
4         automatic int agent_id = i;
5         fork
6             run_agent(agent_id);    // Background thread
7         join_none
8     end
9
10    // Continue immediately, do other setup...
11    configure_dut();
12    enable_clocks();
13
14    // Now wait for ALL agent threads to finish
15    wait fork;
16    $display("All agents completed!");
17
18    // Check coverage
19    assert(cg.get_coverage() > 95.0);
20    $finish;
21 end
```

Listing 10.3: wait fork usage

11.1 UVM Testbench Structure

Q40. Describe the layered UVM testbench architecture.

Answer:

A standard UVM testbench is organized in layers:

1. **Test:** Top-level test class. Configures the environment and starts sequences.
2. **Environment (Env):** Contains one or more agents, scoreboards, and coverage collectors.
3. **Agent:** Contains driver, monitor, and sequencer. Can be active (drives DUT) or passive (observes only).
4. **Driver:** Converts sequence items into pin-level transactions on the DUT interface.
5. **Monitor:** Passively observes DUT pins; converts to transactions; broadcasts via analysis port.
6. **Sequencer:** Manages sequence execution and arbitration; connects to driver via TLM.
7. **Scoreboard:** Compares DUT outputs to expected outputs (reference model).
8. **Coverage Collector:** Gathers functional coverage from monitored transactions.

```
1 // Sequence Item
2 class my_seq_item extends uvm_sequence_item;
```

```

3   'uvm_object_utils(my_seq_item)
4   rand logic [31:0]  addr;
5   rand logic [31:0]  data;
6   rand logic          write;
7
8   constraint valid_addr { addr[1:0] == 2'b00; } // 4B align
9 endclass
10
11 // Driver
12 class my_driver extends uvm_driver #(my_seq_item);
13   'uvm_component_utils(my_driver)
14   virtual my_if vif;
15
16   task run_phase(uvm_phase phase);
17     forever begin
18       my_seq_item item;
19       seq_item_port.get_next_item(item);
20       drive_item(item);
21       seq_item_port.item_done();
22     end
23   endtask
24
25   task drive_item(my_seq_item item);
26     @(posedge vif.clk);
27     vif.addr    <= item.addr;
28     vif.data    <= item.data;
29     vif.write   <= item.write;
30     vif.valid   <= 1;
31     @(posedge vif.clk);
32     vif.valid   <= 0;
33   endtask
34 endclass
35
36 // Test
37 class my_test extends uvm_test;
38   'uvm_component_utils(my_test)
39   my_env env;
40
41   task run_phase(uvm_phase phase);
42     my_sequence seq = my_sequence::type_id::create("seq");
43     phase.raise_objection(this);
44     seq.start(env.agent.sequencer);
45     phase.drop_objection(this);
46   endtask
47 endclass

```

Listing 11.1: UVM class hierarchy skeleton

Q41. What is the UVM factory and why is it important?

Answer:

The **UVM factory** is a registration and override mechanism that allows test writers to substitute one class with another (derived class) without modifying the component hierarchy. This enables:

- **Type overrides:** Replace a class globally (`type_id::create`)
- **Instance overrides:** Replace a specific instance only
- **Configuration independence:** Tests can vary behavior without changing env code

```

1 // Register classes with factory
2 class base_driver extends uvm_driver#(my_item);
3     'uvm_component_utils(base_driver) // Factory registration
4     // ...
5 endclass
6
7 class error_driver extends base_driver;
8     'uvm_component_utils(error_driver)
9     // Override behavior to inject errors
10    task drive_item(my_item item);
11        item.data ^= 32'hDEAD_BEEF; // Corrupt data!
12        super.drive_item(item);
13    endtask
14 endclass
15
16 // In test: override without changing env code
17 class error_test extends uvm_test;
18     function void build_phase(uvm_phase phase);
19         // Replace all base_driver instances with error_driver
20         base_driver::type_id::set_type_override(
21             error_driver::get_type());
22         super.build_phase(phase);
23     endfunction
24 endclass

```

Listing 11.2: UVM factory registration and override

Q42. What is the TLM (Transaction Level Modeling) in UVM?

Answer:

TLM defines standardized communication interfaces between UVM components using:

- **uvm_analysis_port:** One-to-many broadcast (monitor → scoreboard/coverage)

- `uvm_blocking_put_port/export`: Blocking producer-consumer
- `uvm_tlm_analysis_fifo`: Buffers items between components
- **Sequencer-Driver TLM**: `seq_item_port.get_next_item()` / `item_done()`

```

1  // Monitor broadcasts to all subscribers
2  class my_monitor extends uvm_monitor;
3      uvm_analysis_port #(my_item) ap;  // 1-to-many port
4
5      function void build_phase(uvm_phase phase);
6          ap = new("ap", this);
7      endfunction
8
9      task run_phase(uvm_phase phase);
10         my_item item;
11         forever begin
12             // Capture transaction from interface
13             item = my_item::type_id::create("item");
14             @(posedge vif.clk iff vif.valid);
15             item.addr = vif.addr;
16             item.data = vif.data;
17             ap.write(item);  // Broadcast to all subscribers
18         end
19     endtask
20 endclass
21
22 // Scoreboard subscribes via analysis export
23 class my_scoreboard extends uvm_scoreboard;
24     uvm_analysis_imp #(my_item, my_scoreboard) analysis_export
25         ;
26
27     function void write(my_item item);
28         // Compare item against reference model
29         check_item(item);
30     endfunction
31 endclass

```

Listing 11.3: TLM analysis port usage

12.1 RTL and Testbench Debugging

Q43. What is an X-state propagation issue and how do you debug it?

Answer:

X-propagation occurs when an **unknown (X) value** spreads through the design, typically from:

- Uninitialized flip-flops (before reset completes)
- Bus contention (multiple drivers)
- Unresolved don't-care logic
- Missing reset logic

Debug strategies:

1. Use `$isunknown(signal)` to detect X in assertions
2. Check waveforms for X-to-0/1 transitions indicating uninitialized state
3. Enable `xprop` simulation mode (Questa/VCS)
4. Use `casex/casez` carefully — they mask X values
5. Ensure all FFs have an explicit reset path

```
1 // Detect X on output
2 property no_x_on_data_out;
3   @(posedge clk) disable iff (!rst_n)
4     valid_out |-> !$isunknown(data_out);
```



```

5 endproperty
6 assert property (no_x_on_data_out)
7     else $error("X detected on data_out!");
8
9 // Safe case statement (avoid caseX with real signals)
10 always_comb begin
11     unique case (state)
12         IDLE: next_state = (req) ? ACTIVE : IDLE;
13         ACTIVE: next_state = (done) ? IDLE : ACTIVE;
14         default: next_state = IDLE; // Catches X states in sim
15     endcase
16 end
17
18 // Checking for X in testbench
19 if ($isunknown(dut.output_bus)) begin
20     $error("[%0t] Unknown value on output_bus!", $time);
21 end

```

Listing 12.1: X-detection in assertions and code

Q44. What is a race condition in simulation and how do you avoid it?**Answer:**

A **race condition** occurs when two events at the same simulation time produce different results depending on evaluation order. Common in testbenches:

- Driving signals at the same clock edge the DUT samples them
- Using blocking assignments in sequential logic
- Missing automatic on tasks used in fork/join

Avoid by: using clocking blocks (drive with output skew), using `<=` in sequential always blocks, and using **program** blocks for testbenches.

```

1 // RACE: TB drives at posedge, DUT samples at posedge
2 initial begin
3     @(posedge clk);
4     data = 8'hAB; // Racing with DUT's always_ff!
5 end
6
7 // FIX 1: Drive slightly after clock edge
8 initial begin
9     @(posedge clk);
10    #1; // Small delay avoids race
11    data = 8'hAB;
12 end
13
14 // FIX 2: Use clocking block (recommended in UVM)

```

```

15 initial begin
16     @(cb);           // Sync to clocking block
17     cb.data <= 8'hAB; // Driven per output skew setting
18 end
19
20 // FIX 3: Drive on negedge if DUT samples on posedge
21 initial begin
22     @(negedge clk);
23     data = 8'hAB;    // Data stable before next posedge
24 end

```

Listing 12.2: Race condition example and fix

Q45. How do you debug a failing UVM test? Walk through your methodology.

Answer:

A systematic debugging flow for UVM test failures:

1. **Read the error message:** Is it an assertion, scoreboard mismatch, timeout, or fatal?
2. **Check timestamps:** Isolate the exact simulation time of failure
3. **Enable UVM verbosity:** Increase to UVM_HIGH or UVM_FULL
4. **Dump waveforms:** Open in GTKWave/DVE/Verdi; check control signals
5. **Trace back the transaction:** Find the stimulus that caused the failure
6. **Check the scoreboard:** Compare predicted vs actual values
7. **Isolate:** Reduce to a minimal failing test case
8. **Check reset:** Ensure proper reset assertion/deassertion

```

1 // Increase verbosity from command line
2 // % vsim -sv_lib uvm top_tb +UVM_VERBOSITY=UVM_HIGH
3
4 // Runtime verbosity change in test
5 class debug_test extends my_test;
6     task run_phase(uvm_phase phase);
7         uvm_top.set_report_verbosity_level_hier(UVM_HIGH);
8         super.run_phase(phase);
9     endtask
10 endclass
11
12 // UVM messaging
13 'uvm_info("MY_COMP",
14     $sformatf("Transaction: addr=%h data=%h", addr, data),

```

```

15     UVM_MEDIUM)
16
17     'uvm_error("SCOREBOARD",
18         $sformatf("MISMATCH: exp=%h got=%h", expected, actual))
19
20     'uvm_fatal("CONFIG", "Interface not set in config_db!")
21
22     // Check config_db
23     if (!uvm_config_db#(virtual my_if)::get(
24         this, "", "vif", vif)) begin
25         'uvm_fatal("NO_VIF", "Virtual interface not found!")
26     end

```

Listing 12.3: UVM debug utilities

Q46. What causes a simulation to hang and how do you debug it?**Answer:**

Simulation hangs (runs forever without advancing or completing) are caused by:

- **Infinite loop:** forever without a timing control
- **Deadlock:** Two threads each waiting for the other (circular dependency)
- **Mailbox/semaphore starvation:** A thread waiting on `get()` that never receives data
- **Missing objection:** Phase does not end because `drop_objection()` was never called
- **Zero-delay loop:** Combinational loop causing infinite delta cycles

```

1  // BUG: Infinite loop without time advance
2  always begin
3      process_data(); // No #, @, or wait -- simulator hangs!
4  end
5
6  // FIX: Add timing control
7  always begin
8      @(posedge clk);
9      process_data();
10 end
11
12 // BUG: Deadlock (circular wait)
13 // Thread A: sema_a.get(); sema_b.get();
14 // Thread B: sema_b.get(); sema_a.get(); <- DEADLOCK
15
16 // FIX: Consistent lock ordering
17 // Thread A: sema_a.get(); sema_b.get();

```

```

18 // Thread B: sema_a.get(); sema_b.get();  <- Same order
19
20 // Watchdog timer to detect hangs
21 initial begin
22     #MAX_TEST_TIME;
23     'uvm_fatal("WATCHDOG",
24         "Simulation exceeded maximum time limit!")
25 end
26
27 // UVM objection timeout
28 function void phase_timeout(uvm_phase phase);
29     phase.phase_done.set_drain_time(this, 1us);
30 endfunction

```

Listing 12.4: Common hang causes and fixes

Q47. Explain the significance of unique, unique0, and priority case modifiers.

Answer:

These modifiers add synthesis directives and simulation checking to `case` statements:

- **unique case:** Asserts that *exactly one* branch matches and all possible values are covered. Simulation warning if zero or multiple branches match. Implies parallel (priority-free) hardware.
- **unique0 case:** Same but allows zero matching branches (no warning for no match). Used when some inputs are impossible/don't care.
- **priority case:** Asserts that *at least one* branch always matches. Implies priority encoder hardware. Warning if no branch matches.

```

1 // unique: exactly one match expected (parallel mux)
2 unique case (sel)
3     2'b00: y = a;
4     2'b01: y = b;
5     2'b10: y = c;
6     2'b11: y = d;
7     // No default needed: unique asserts all cases covered
8 endcase
9
10 // priority: first match wins (priority encoder)
11 priority case (1'b1)
12     irq[0]: handle_irq0(); // Highest priority
13     irq[1]: handle_irq1();
14     irq[2]: handle_irq2();
15     // Simulation warning if no irq is active
16 endcase

```

```

17
18 // unique0: parallel, but 'no match' is valid
19 unique0 case (opcode)
20     ADD: result = a + b;
21     SUB: result = a - b;
22     AND: result = a & b;
23     // No warning if opcode is NOP (not listed)
24 endcase

```

Listing 12.5: unique, unique0, priority case

Q48. What are the differences between \$display, \$write, \$monitor, and \$strobe?

Answer:

- **\$display:** Prints immediately when executed in the Active region. Adds a newline. Most common.
- **\$write:** Same as \$display but *no* newline.
- **\$monitor:** Triggers *once per time step* whenever any of its arguments changes value. Evaluated in the Postponed region. Only one \$monitor can be active at a time.
- **\$strobe:** Prints *at the end of the current time step* (Postponed region) after all non-blocking assignments have settled. Shows final stable values.

```

1 logic [3:0] a, b;
2
3 initial begin
4     a = 4; b = 3;
5
6     // $display: shows value AT THIS MOMENT
7     $display("a=%0d b=%0d", a, b); // 4 3
8
9     a <= 10; // NBA: not updated yet
10
11    // $display still shows old value in Active region
12    $display("After NBA: a=%0d", a); // Still 4!
13
14    // $strobe: shows value after NBA updates
15    $strobe("Strobe: a=%0d", a); // Shows 10 (correct)
16 end
17
18 // $monitor: automatic tracking
19 initial $monitor("[%0t] a=%0d b=%0d", $time, a, b);
20 // Prints whenever a or b changes

```

Listing 12.6: Display system tasks

13.1 Mixed Concept Questions

Q49. What is the difference between program block and module in SystemVerilog testbenches?

Answer:

The `program` block was introduced to address race conditions between testbench and DUT:

- **Scheduling:** `program` executes in the Re-Active region (after the DUT's NBA region), avoiding races by default
- **No always blocks:** Only `initial` blocks allowed in programs
- **Automatic:** All variables in program are automatic by default
- **End behavior:** When the last `initial` block finishes, simulation ends

In practice, modern UVM environments use `module`-based testbenches with clocking blocks instead of `program` blocks, due to tool compatibility.

Q50. How does `uvm_config_db` work and what are common pitfalls?

Answer:

`uvm_config_db` is a global configuration database using a hierarchical string key to pass objects, virtual interfaces, and settings between components. `set()` stores, `get()` retrieves. Common pitfalls:

1. Wrong path string in `set/get`
2. Calling `get` before `set` (wrong phase ordering)
3. Type mismatch between `set` and `get` parameter
4. Missing `uvm_fatal` on failed `get`

```

1 // In top-level module: set virtual interface
2 initial begin
3     uvm_config_db #(virtual my_if)::set(
4         null,          // Context (null = top-level)
5         "uvm_test_top.env.agent", // Path
6         "vif",         // Field name
7         dut_if);       // Value
8
9     run_test("my_test");
10 end
11
12 // In driver: get virtual interface
13 class my_driver extends uvm_driver #(my_item);
14     virtual my_if vif;
15
16     function void build_phase(uvm_phase phase);
17         if (!uvm_config_db #(virtual my_if)::get(
18             this,      // Component context
19             "",        // Instance path (empty = this component)
20             "vif",     // Field name
21             vif))      // Retrieved value
22             `uvm_fatal("NO_VIF", "Cannot get virtual interface!")
23     endfunction
24 endclass

```

Listing 13.1: uvm_config_db usage

Q51. What is an event in SystemVerilog? Compare it with Verilog events.

Answer:

SV `event` is an enhanced synchronization primitive over Verilog:

- Can be passed by reference between tasks and classes
- `->>` triggers an event; `@(event)` waits for it
- `triggered` property: non-blocking check if event occurred in current time step
- Can be merged (assigned): `e1 = e2` makes both reference the same event


```
1 event done_e;
2 event error_e;
3
4 // Thread 1: waits for event
5 initial begin
6     $display("Waiting for done...");
7     @done_e;                      // Wait for trigger
8     $display("Done received!");
9 end
10
11 // Thread 2: triggers event
12 initial begin
13     #100ns;
14     ->done_e;                    // Trigger event
15 end
16
17 // Using .triggered (non-blocking check)
18 initial begin
19     @(posedge clk);
20     if (done_e.triggered)
21         $display("Event already triggered this timestep");
22 end
23
24 // Event in class (pass by reference)
25 class my_agent;
26     event trans_done;
27     task wait_for_done();
28         @trans_done;
29     endtask
30     task signal_done();
31         ->trans_done;
32     endtask
33 endclass
```

Listing 13.2: SystemVerilog events

Q52. What is the difference between static and automatic class properties?

Answer:

- **static class property:** Shared across ALL instances of the class. One copy in memory. Used for counters, instance IDs, shared resources.
- **Normal (non-static) property:** Each object has its own independent copy.
- **static method:** Can be called without a class instance handle. Can only

access static properties.

```

1 class transaction;
2     static int obj_count = 0;      // Shared by all instances
3     int id;                       // Per-instance
4     rand logic [31:0] data;
5
6     function new();
7         obj_count++;              // Increment shared counter
8         id = obj_count;           // Assign unique ID
9     endfunction
10
11     static function int get_count();
12         return obj_count;
13     endfunction
14 endclass
15
16 transaction t1 = new(); // obj_count = 1, t1.id = 1
17 transaction t2 = new(); // obj_count = 2, t2.id = 2
18 transaction t3 = new(); // obj_count = 3, t3.id = 3
19
20 $display("Total created: %0d", transaction::get_count()); //
    3
21 $display("t2 ID: %0d", t2.id); // 2 (per-instance)

```

Listing 13.3: static class property

Q53. What are the four UVM phases and their purposes?

Answer:

UVM phases provide a standardized lifecycle for testbench components:

1. **Build phase:** Component creation (**new**, **create**), configuration retrieval, port creation. Top-down execution.
2. **Connect phase:** TLM port connections between components. Bottom-up execution.
3. **Run phase:** Active simulation (**time-consuming**). All components run concurrently. Controlled by objections.
4. **Report phase:** Print final statistics, coverage, pass/fail summary. Bottom-up.

Additional: `start_of_simulation`, `end_of_elaboration`, `extract`, `check`, `final` phases also exist.

```

1 class my_component extends uvm_component;

```

```

2   'uvm_component_utils(my_component)
3
4   // Build: create sub-components (top-down)
5   function void build_phase(uvm_phase phase);
6       super.build_phase(phase);
7       // Create child components here
8   endfunction
9
10  // Connect: wire up TLM ports (bottom-up)
11  function void connect_phase(uvm_phase phase);
12      // monitor.ap.connect(scoreboard.analysis_export);
13  endfunction
14
15  // Run: time-consuming simulation
16  task run_phase(uvm_phase phase);
17      phase.raise_objection(this);
18      // Execute sequences, drive stimulus...
19      #1000ns;
20      phase.drop_objection(this);
21  endtask
22
23  // Report: final summary (bottom-up)
24  function void report_phase(uvm_phase phase);
25      'uvm_info(get_name(),
26          $sformatf("Transactions processed: %0d", tx_count),
27          UVM_LOW)
28  endfunction
29  endclass

```

Listing 13.4: UVM phase skeleton

Q54. What is the foreach loop and how does it differ from for?

Answer:

foreach automatically iterates over all elements of an array (static, dynamic, associative, or queue) without needing to know the size or indices explicitly. The variable declared in **foreach** is local to the loop.

```

1  // Static array
2  logic [7:0] mem [256];
3  foreach (mem[i]) mem[i] = '0;    // Zero all elements
4
5  // Multi-dimensional
6  logic [3:0][7:0] matrix [4][4];
7  foreach (matrix[i,j]) matrix[i][j] = i * 4 + j;
8
9  // Dynamic array
10 int dyn[] = new[10];

```

```

11 foreach (dyn[k]) dyn[k] = k * k;
12
13 // Associative array
14 int score [string];
15 score["Alice"] = 95; score["Bob"] = 87;
16 foreach (score[name])
17     $display("%s scored %0d", name, score[name]);
18
19 // Queue
20 int q[$] = {10, 20, 30, 40};
21 foreach (q[i]) $display("q[%0d] = %0d", i, q[i]);
22
23 // Equivalent for loop (requires knowing size)
24 for (int i = 0; i < dyn.size(); i++)
25     $display("%0d", dyn[i]);

```

Listing 13.5: foreach loop usage

Q55. Explain how to implement a parameterized class in SystemVerilog.

Answer:

SystemVerilog classes support **parameter types**, enabling generic, reusable data structures (like C++ templates). The parameter can be a type, integer, or string.

```

1 // Generic FIFO class
2 class generic_fifo #(type T = int, int DEPTH = 8);
3     T    buffer [DEPTH];
4     int   head = 0, tail = 0, count = 0;
5
6     function void push(input T item);
7         if (count == DEPTH) $error("FIFO overflow!");
8         else begin
9             buffer[tail] = item;
10            tail = (tail + 1) % DEPTH;
11            count++;
12        end
13    endfunction
14
15    function T pop();
16        T item;
17        if (count == 0) $error("FIFO underflow!");
18        else begin
19            item = buffer[head];
20            head = (head + 1) % DEPTH;
21            count--;
22        end
23        return item;
24    endfunction

```

```

25
26     function bit is_empty(); return (count == 0); endfunction
27     function bit is_full();  return (count == DEPTH);
        endfunction
28 endclass
29
30 // Instantiate with different types and depths
31 generic_fifo #(int, 16)          int_fifo  = new();
32 generic_fifo #(logic [31:0], 32) data_fifo = new();
33
34 // Custom type in FIFO
35 typedef struct packed { logic [31:0] addr, data; } txn_t;
36 generic_fifo #(txn_t, 4) txn_fifo = new();

```

Listing 13.6: Parameterized class example

Q56. What is a virtual interface and why is it essential in UVM?**Answer:**

A **virtual interface** is a variable that holds a reference to a physical interface instance. Since UVM classes are dynamic objects not connected to the hardware hierarchy, they cannot directly access module-scope signals. The virtual interface bridges this gap:

1. Physical interface is instantiated in the static top-level module
2. The reference is stored in `uvm_config_db`
3. UVM components (driver, monitor) retrieve the reference via `uvm_config_db::get`
4. The UVM component uses the virtual interface to drive/sample DUT signals

```

1 // Interface definition
2 interface my_if (input logic clk, rst_n);
3     logic [7:0] data;
4     logic      valid, ready;
5 endinterface
6
7 // Top-level module: instantiate physical interface
8 module top;
9     logic clk, rst_n;
10    my_if dut_if (.clk(clk), .rst_n(rst_n));
11
12    // Pass to UVM via config_db
13    initial begin
14        uvm_config_db #(virtual my_if)::set(
15            null, "uvm_test_top.*", "vif", dut_if);
16        run_test();
17    end

```

```

18 endmodule
19
20 // UVM Driver: use virtual interface
21 class my_driver extends uvm_driver #(my_item);
22     virtual my_if vif; // Virtual interface handle
23
24     function void build_phase(uvm_phase phase);
25         if (!uvm_config_db #(virtual my_if)::get(
26             this, "", "vif", vif))
27             `uvm_fatal("NOVIF", "Virtual interface not found")
28     endfunction
29
30     task run_phase(uvm_phase phase);
31         my_item item;
32         forever begin
33             seq_item_port.get_next_item(item);
34             @(posedge vif.clk);
35             vif.data <= item.data; // Drive through virtual
36                                     iface
37             vif.valid <= 1;
38             seq_item_port.item_done();
39         end
40     endtask
41 endclass

```

Listing 13.7: Virtual interface setup

Q57. What are the key differences between `assert`, `assume`, and `cover` in SVA?

Answer:

- **assert property:** The property *must* be true. Failure is reported as an error/fatal. Used to verify DUT behavior.
- **assume property:** The property is *assumed* to be true. In formal verification, it constrains the input space. In simulation, acts like an `assert` to verify stimulus validity.
- **cover property:** Checks that the property *can* be true at least once. Used for coverage — ensures that a scenario was exercised.

```

1 // Assert: DUT must obey this rule
2 assert property (@(posedge clk) disable iff (!rst_n)
3     req |-> ##[1:4] ack)
4     else $error("Protocol violation: ACK not received");
5
6 // Assume: Constrain environment inputs (for formal)

```

```
7  assume property (@(posedge clk)
8      !($isunknown(data_in)) ) // Input is never X
9
10 // Cover: Verify this scenario was reached
11 cover property (@(posedge clk)
12     req ##1 !ack ##1 ack) // ack delayed by 2 cycles
13     $display("2-cycle ack scenario covered");
14
15 // Named properties for reuse
16 property back_to_back;
17     @(posedge clk) req ##1 req;
18 endproperty
19
20 assert property (back_to_back);
21 cover  property (back_to_back);
```

Listing 13.8: assert, assume, cover in SVA

1

Top 10 Most-Asked SystemVerilog Interview Topics:

1. **Blocking vs Non-Blocking Assignments** — Always explain race conditions
2. **4-State vs 2-State types** — `logic` vs `bit`, X/Z handling
3. **`always_ff` / `always_comb`** — Preferred over generic `always`
4. **OOP: `virtual`, `pure virtual`, `$cast`** — UVM foundation
5. **`rand`/`randc` + `constraints`** — Constraint-driven verification
6. **SVA: `assert`/`assume`/`cover`, `—i` vs `==i`** — Property checking
7. **`fork`/`join` variants** — Concurrent process management
8. **UVM Architecture** — `Test`→`Env`→`Agent`→`Driver`/`Monitor`/`Sequencer`
9. **Mailbox/Semaphore** — IPC primitives
10. **Functional vs Code Coverage** — Know when to use each

Good luck in your interviews!

“Verification is not about finding bugs — it’s about building confidence.”
